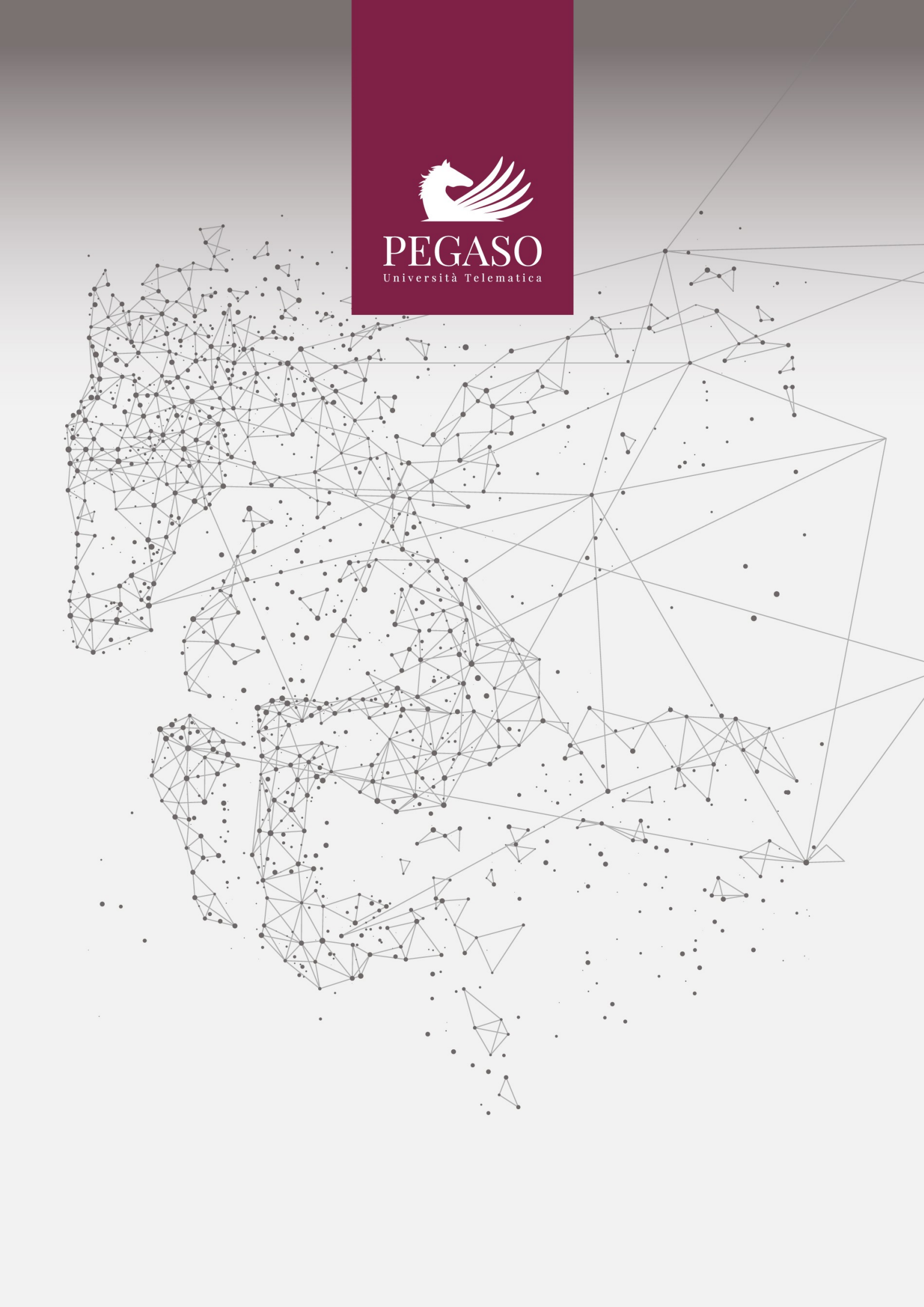

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Indice

1. INTRODUZIONE ALLA PROGRAMMAZIONE OBJECT-ORIENTED IN JAVA.....3

2. ELEMENTI BASE DELLA PROGRAMMAZIONE OBJECT-ORIENTED IN JAVA.....4

3. INCAPSULAMENTO 6

4. VANTAGGI DELL'INCAPSULAMENTO.....10

5. VISIBILITÀ 12

BIBLIOGRAFIA 19

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

1. Introduzione alla programmazione Object-Oriented in Java

Con questa lezione inizia una serie di spiegazioni dedicate al supporto che Java offre alla programmazione ad oggetti.

Tramite i costrutti di base come ad esempio i cicli e per le condizioni è assolutamente possibile risolvere problemi algoritmici scrivendo del codice Java corretto.

Tuttavia senza nulla togliere all'importanza di risolvere problemi algoritmici (che richiedono indubbie qualità logiche), nella vita reale ci si trova spesso a dover scrivere software per risolvere problemi, quali ad esempio la gestione delle fatture in un programma gestionale, per risulta essenziale usare strumenti di programmazione più sofisticati per essere in grado di creare un programma da zero con qualità sufficiente.

In altre parole, occorrono delle nozioni e dei costrutti necessari per essere in grado di realizzare veri e propri progetti o prodotti software anche da commercializzare e su cui costruire un'attività di business.

In questa visione più evoluta di programmazione entrano in gioco le seguenti domande:

- Come organizzare un programma da zero e di qualità ovvero affidabile e manutenibile ed estendibile?
- Quanti classi creare?
- Come si dovrebbero chiamare queste classi?
- Che metodi dovranno definire?
- Come creare un programma capace di evolversi senza ritoccare ogni volta parti di codice già scritte?

L'ultimo punto è particolarmente importante nell'ottica di sviluppare prodotti software in quanto rimettere mano al codice già scritto per espanderlo significa esporsi al rischio che le funzionalità già implementate non funzionano più come dovrebbero generando evidentemente dei costi (es. testing aggiuntivo per ritestare il sistema nella funzionalità già rilasciate al cliente).

A queste e a altre domande risponde la programmazione orientata agli oggetti che permette di gestire la complessità nella programmazione nel momento in ci andiamo a costruire sistemi software per risolvere problemi reali legati alla realtà che ci circonda.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

2. Elementi base della programmazione Object-Oriented in Java

La programmazione orientata agli oggetti è una filosofia di progettazione del software.

Essa si basa su concetti esistenti nel mondo reale con i quali abbiamo a che fare nella vita di tutti i giorni.

Il concetto di classe, di oggetto e astrazione ci aiutano a rappresentare la realtà che ci circonda nel contesto di un programma software ed anche gestire la sua complessità così come avviene nella vita di tutti i giorni.

L'**incapsulamento** è una delle 3 proprietà chiavi della programmazione orientata agli oggetti.

Consente di creare programmi composti da classi, che nascondono dettagli di implementazione dietro interfacce pubbliche.

Quest'ultime regoleranno la comunicazione fra gli oggetti che fanno parte del sistema, comunicazioni che permettono quelle interazioni che costituiscono il funzionamento in sé del programma.

L'incapsulamento è legato al concetto di visibilità dei componenti software (attributi e metodi) visibilità che in Java è regolata essenzialmente dai modificatori di accesso, dal riferimento this, dai package, dal modificatore static.

L'incapsulamento è legato anche ad altri due concetti importanti nell'object oriented: riuso ed astrazione.

Quest'ultimi due paradigmi sono in genere meno esaltati in Java non perché meno significativi e utili ma perché non sono specifici della programmazione orientata agli oggetti in quanto esistevano anche precedentemente.

Ad ogni modo, l'incapsulamento è ispirato dall'**astrazione** e favorisce il **riuso** del codice.

L'astrazione consiste nel processo che porta a concentrarsi sulle proprietà rilevanti di un'entità in relazione al problema che stiamo risolvendo, ignorando i dettagli inessenziali.

L'astrazione nell'approccio orientato agli oggetti entra in gioco nell'individuazione e definizione delle classi di oggetti e relative proprietà.

La classe è infatti frutto di un'astrazione di un concetto del mondo reale che, da un punto software si esprime attraverso un insieme di dati e un insieme di operazioni che agiscono su quei dati.

I dati e le operazioni che si sceglie di definire nel contesto del problema sono frutto del processo di astrazione. L'incapsulamento va oltre andando a nascondere i dettagli della implementazione della classe dietro interfacce pubbliche attraverso le quali si interagisce con l'oggetto.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Il riuso è supportato dall'astrazione e dall'incapsulamento in quanto le classi che descrivono oggetti ben incapsulati sono facili da riutilizzare in applicazioni differenti.

L'astrazione e una opportuna specifica dell'interfaccia pubblica dell'oggetto può essere invocato in più parti del codice ed è possibile modificare l'implementazione di un oggetto senza influenzare le applicazioni che lo usano (fintantoché l'interfaccia resta inalterata).

Il riuso è anche promosso intrinsicamente anche dagli altri due paradigmi della programmazione ad oggetti ovvero l'ereditarietà e il polimorfismo, altri due concetti chiave nella programmazione ad oggetti (e in Java) che vanno a sfruttare caratteristiche comuni fra gli oggetti delle classi (ereditarietà) oppure a permettere di differenziare il comportamento degli oggetti in modo dinamico (polimorfismo).

Il linguaggio Java rispetto ad altri nasce proprio come linguaggio di programmazione orientato agli oggetti, quindi con costrutti specifici e dedicati alla programmazione ad oggetti (anche forzandola nell'applicazione dei suoi principi).

A differenza di altri linguaggi di programmazione orientati agli oggetti, Java definisce in modo estremamente chiaro i concetti sopra menzionati.

Anche programmatori che si ritengono esperti in altri linguaggi orientati agli oggetti come il C++ studiando Java potrebbero comprendere in modo più profondo questi paradigmi che permettono di creare programmi più semplici da evolvere e manutenere, che riusano parti di codice ben definite e che facilitano l'interazione tra il programmazione e il codice.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

3. Incapsulamento

L'incapsulamento è la proprietà per cui un oggetto contiene ("incapsula") al suo interno gli attributi (dati) e le operazioni (procedure) che agiscono sui dati stessi.

Tramite l'incapsulamento un oggetto appartenente ad una classe è considerato costituito da due parti distinte:

- un'interfaccia visibile all'esterno, interfaccia pubblica, che serve per l'interazione dell'oggetto con l'esterno.
- una parte interna nascosta all'esterno e costituita dai dati e dai metodi (che operano su tali dati) dando luogo così ad un certo comportamento.

La filosofia dell'incapsulamento si basa sull'accesso controllato ai dati mediante metodi speciali che possano prevenirne l'utilizzo non consono e gli errori.

L'incapsulamento è leggermente diverso dal concetto di astrazione.

L'incapsulamento corrisponde a separare l'interfaccia dall'implementazione degli oggetti.

L'astrazione invece si riferisce al processo che porta ad estrarre le proprietà rilevanti di un'entità, ignorando i dettagli inessenziali.

Per applicare l'incapsulamento ad una classe occorre:

- Dichiarare privati gli attributi al fine di renderli inaccessibili al di fuori dalla classe stessa. A tale scopo si usa il modificatore private.
- Creare dei metodi pubblici (dichiarati con il modificatore public) che permettono l'accesso agli attributi anche da altre classi in maniera controllata. Tali metodi potrebbero realizzare controlli prima di confermare l'accesso ai dati privati evitando che essi assumano valori non validi.

I metodi pubblici nel loro insieme costituiscono l'interfaccia pubblica della classe perché visibili (e invocabili) dall'esterno, garantendo un accesso appropriato e al contempo semplice, chiaro e indipendente dall'implementazione interna.

In particolare l'interfaccia pubblica non è costituita dalla implementazione dei metodi «interni» in quanto questi non sono visibili né modificabili all'esterno e quindi fanno parte della implementazione interna insieme ai dati. Quest'ultimi sono definiti con parola «private».

Si consideri l'esempio di codice Java mostrato in Fig. 1.

Supponiamo di voler scrivere un'applicazione che utilizza la classe Data, la quale astrae in maniera semplice il concetto di data. La classe data ha 3 variabili di istanza: giorno, mese, anno.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Dichiariamo la classe Data pubblica per renderla utilizzabile da altre classi e la definiamo in particolare anche con le variabili di istanza pubbliche.

Immaginiamo di usare tale classe da un'altra classe oppure dal main di un'applicazione software dove serve istanziare un oggetto di tipo Data inizializzato al valore 14 aprile 2004. Per fare questo inizializziamo le variabili di istanza giorno, mese, anno rispettivamente a 14, 4, 2004.

```
…
public class Data {
        public int giorno;
        public int mese;
        public int anno;
} \\ definizione della classe in Java
…
\\ uso della classe nel programma con assegnazione diretta di valori
Data unaData = new Data();
unaData.giorno = 14;
unaData.mese = 4;
unaData.anno = 2004;
…
\\ uso della classe nel programma con assegnazione da interfaccia
grafica
Data unaData = new Data();
unaData.giorno = interfaccia.dammiGiornoInserito();
unaData.mese = interfaccia.dammiMeseInserito();
unaData.anno = interfaccia.dammiAnnoInserito();
\\ e se i valori inseriti fossero 32, 13, 2087?
```

Fig. 1: Esempio di classe Data senza incapsulamento

Qual è il problema? Il programma così fatto compilerebbe senza errori e funzionerebbe correttaemente.

Quindi sembrerebbe nessun problema. Supponiamo però che l'applicazione permetta all'utente di inserire la data tramite un'interfaccia grafica.

In tal caso l'inizializzazione avviene invocando dei metodi denominati dammiGiornoInserito, dammiMeseInserito, dammiAnnoInserito dell'oggetto interfaccia, metodi che restituiscono un numero intero inserito dall'utente mediante l'interfaccia grafica dell'applicazione.

Supponiamo che i valori inseriti siano 32 per il giorno, 13 per il mese e 2087 per l'anno; ecco che i problemi di questo codice cominciano ad emergere.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Come possiamo evitare che l'utente inserisca dei dati errati ovvero che delle variabili di istanza pubbliche assumano valore non opportuni?

Ci potrebbero essere le seguenti due opzioni per controllare l'accesso alle variabili di istanza.

1- si potrebbe limitare la possibilità degli inserimenti sull'interfaccia grafica all'utente e quindi prevedere dei controlli nella classe corrispondente all'oggetto interfaccia.

2- si potrebbe delegare ai metodi dammiGiornoInserito, dammiMeseInserito, dammiAnnoInserito dell'oggetto interfaccia i controlli necessari per una corretta impostazione della data.

In entrambi i casi il problema sarebbe risolto ma solo nel caso in cui l'impostazione della data avvenga sempre e comunque tramite l'interfaccia grafica e quindi solo per quella specifica funzionalità.

Ma se volessimo riutilizzare in un'altra funzione la classe Data senza riutilizzare la stessa interfaccia grafica, saremmo costretti a scrivere nuovamente del codice di controllo per gestire il problema.

Come procedere allora? Facendo uso dell'incapsulamento e definendo diversamente la classe Data con gli attributi giorno, mese, anno definiti private e poi dei metodi public per accedere a tali dati sia in lettura che in scrittura come mostrato in Fig. 2.

```
…
public class Data {
        private int giorno, mese, anno;
        public void setGiorno (int g) {
                if (g>0 && g<=31) giorno=g;
                else System.out.println(«Giorno non valido»)
        }
        public int getGiorno () {
                return giorno;
        }
        public void setMese (int m) {
                if (m>0 && m<=12) mese=m;
                else System.out.println(«Mese non valido»)
        }
        public int getMese () {
                return mese;
        }
        public int setAnno (int a) {
                anno=a;
        }
        public int getAnno () {
                return anno;
        }
```

Fig. 2: Esempio di classe Data con incapsulamento

Questi metodi seguono una convenzione utilizzata anche nella libreria standard. In presenza di un variabile di istanza privata, chiamiamo questi metodi con la sintassi *setNomeVariabile* e *getNomeVariabile*.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Esiste un'unica eccezione a questa convenzione, quando incontriamo una variabile di tipo booleano. In questo caso il metodo *getNomeVariabile* si chiamerà *isNomeVariabile*.

Anche se all'inizio potrà risultare noioso (la seconda versione della classe Data è decisamente più estesa della prima) implementare l'incapsulamento non richiede la grossa inventiva da parte dello sviluppatore.

Non c'è niente di concettuale, solo da definire dei metodi banali. Tuttavia sono da scrivere e possono allungare la lista dei metodi significativamente come in questo caso. Fortunatamente tutti gli IDE fra cui anche EJE offrono strumenti per la generazione automatica dei metodi get e set.

Volendo mostrare in termini UML la differenza dei due casi di classe incapsulata e non si avrebbero le due classe rappresentate in Fig. 3.

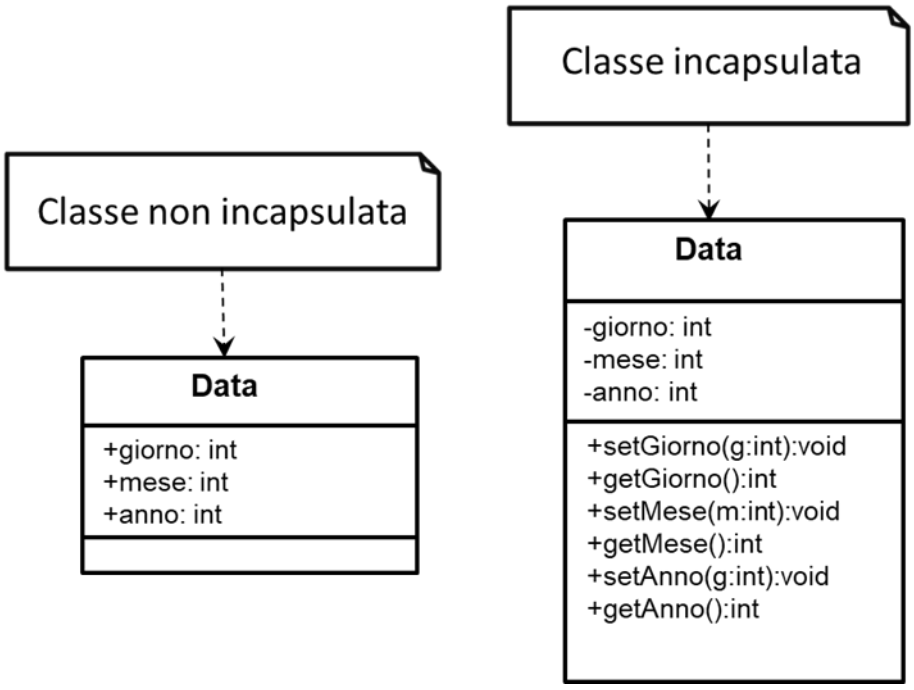


Fig. 3: Diagrammi delle classi per la classe Data non incapsulata (sinistra) e la classe Data incapsulata (destra)

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

4. Vantaggi dell'incapsulamento

Accesso controllato ai dati

Nel momento in cui dichiariamo i dati privati questi non saranno più accessibili mediate l'operatore «dot» a meno che il codice che accede al dato privato non si trovi nella classe che lo ha dichiarato. Questo implica che il codice di uso della classe Data deve modificarsi come mostrato in Fig. 4.

Utilizzando l'interfaccia costituita dai metodi pubblici set e get per accedere ai dati privati, così come quando vogliamo usufruire di uno smartphone siamo costretti ad utilizzare l'interfaccia pubblica costituita dal touchscreen. Infatti i metodi set e get hanno implementazioni che si trovano internamente alla classe Data, e quindi possono accedere ai dati privati, ed anche a metodi.

Si noti che per esempio il metodo setGiorno imposta la variabile giorno con il parametro che gli viene passato solo se risulta compreso tra 1 e 31, altrimenti stampa un messaggio di errore.

Un controllo simile viene fatto anche nel metodo setMese.

Quindi a priori, ogni oggetto Data non può essere impostato con dati non validi per le variabili giorno e mese. Questo implica maggiori opportunità di riuso e robustezza del codice.

Infatti in questo modo una classe che rappresenta una data funzionante, potrebbe essere facilmente utilizzata in programmi completamente diversi.

```
…
Data unaData = new Data();
unaData.setGiorno(14);
unaData.setMese (4);
unaData.setAnno(2004);
…
Data unaData = new Data();
unaData.setGiorno(interfaccia.dammiGiornoInserito);
unaData.setMese(interfaccia.dammiMeseInserito);
unaData.setAnno(interfaccia.dammiAnnoInserito);
```

Fig. 4: Esempi di codice che utilizza Data incapsulata

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Evoluzione del software

Con l'incapsulamento il codice è molto più facile da manutenere e si adatta ai cambiamenti.

Per esempio, il lettore avrà notato che il codice della classe Data mostrato in Fig. 4 risolve solo in parte i problemi della classe Data.

Infatti permetterebbe l'impostazione del giorno al valore 31, anche se per esempio la variabile mese valesse 4, che equivale al mese di aprile che ha solo 30 giorni.

E non risolve nemmeno il problema di febbraio avente solo 28 giorni e negli anni bisestili 29.

La buona notizia è che possiamo far evolvere la classe Data introducendo tutte le migliorie che desideriamo all'interno del codice della classe Data ma non dovremmo cambiare una riga del codice che usa la classe come ad esempio quello mostrato in Fig. 4.

Per esempio, se cambiamo l'implementazione intera del metodo setGiorno nel modo mostrato in Fig.5:

```
…
public void setGiorno (int g) {
                if (g>0 && g<=31 && mese!=4) {
                        giorno=g;
                }
                else {
                        System.out.println(«Giorno non valido»)
                }
        }
…
```

Fig. 5: Controllo di accesso alla variabile giorno

sarà necessario ricompilare solo la classe Data, mentre il codice precedentemente visto che usano l'interfaccia pubblica rimarranno inalterati.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

5. Visibilità

Il concetto di incapsulamento è strettamente connesso al concetto di visibilità del componente software (classe, variabile, metodo) che può essere regolata in diversi modi, in particolare tramite i modificatori di accesso, il reference this, i package, il modificatore static (ci sarebbe anche final ma non è connesso alla visibilità).

I package sono stati analizzati in precedenza.

Vedremo in seguito i modificatori e il reference this.

Il modificatore è una parola chiave (es. public, private, final o static) capace di cambiare il significato di un componente di un'applicazione Java, fra cui classi (e in generale tutti i tipi Java), metodi, variabili d'istanza, variabili locali, attributi di metodi e costruttori, e altri.

Un modificatore sta ad un componente di un'applicazione Java come un aggettivo sta ad un sostantivo nel linguaggio umano. Vedremo in particolare i modificatori di accesso e il modificatore static.

Modificatori di accesso: private**,** public**,** protected

I modificatori di accesso specificano da quali classi è possibile utilizzare una classe, una variabile di istanza o un metodo.

I modificatori public e private vengono meglio definiti come modificatori di accesso in quanto regolano essenzialmente la visibilità e l'accesso ad un componente Java.

Esiste poi un altro modificatore di accesso protected.

Il modificatore public può essere applicato sia ad un membro (variabile di istanza o metodo) di una classe sia ad una classe stessa.

Un membro dichiarato pubblico sarà accessibile da una qualsiasi classe situata in qualsiasi package.

Per l'incapsulamento le variabili di istanza non dovrebbero mai essere dichiarate pubbliche. Una classe dichiarata pubblica sarà anch'essa visibile da un qualsiasi package (a meno di dichiarazioni di moduli).

Il modificatore private limita la visibilità di una variabile di istanza o di un metodo all'interno della classe in cui sono definiti.

Il modificatore private restringe la visibilità di un membro di una classe alla classe stessa.

Esso rappresenta la chiave dell'incapsulamento per le variabili di istanza.

Si usano anche metodi privati che fanno parte dell'implementazione del comportamento e che sono invocati da altri metodi della stessa classe.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Infine diversamente dal modificatore public, il modificatore private non è applicabile alla dichiarazione di una classe.

Il modificatore d'accesso protected è legato anche al concetto di ereditarietà.

Riguarda infatti l'accessibilità dei membri nelle sottoclassi.

Una sottoclasse è una classe che estende un'altra classe ereditandone le caratteristiche ed estendendo e al contempo specializzandone il comportamento, ad esempio la classe Dipendente e la classe Programmatore (il programmatore è un dipendente che svolge delle mansioni particolari).

I metodi pubblici sono ereditati dalla sottoclasse e quindi possono essere usati senza limiti. Lo stesso dicasi per i metodi protetti.

Di conseguenza il modificatore protected definisce per un membro di una classe il grado più accessibile dopo quello definito da public.

Un membro protetto infatti sarà accessibile all'interno delle stesso package, e verrà anche ereditato in tutte le sottoclassi della classe in cui è definito, anche se non appertenenti allo stesso package di dichiarazione. Infatti i membri dichiarati come protected saranno ereditati come fossero pubblici anche nelle sottoclasse esterne al package.

Diversamente dal modificatore public il modificatore protected non è applicabile alla dichiarazione di una classe.

E' possibile non anteporre alcun modificatore di accesso ad un membro (variabile di istanza o metodo) di una classe o relativamente alla classe stessa. Se non si antepone alcun modificatore di accesso ad un membro di una classe esso sarà accessibile solo da classe appartenenti al package dove è definito.

Modificatore static

Altro modificatore connesso alla visibilità è static. Questo modificatore si applica ai metodi e attributi (ovvero variabili di istanza) di una classe e sta ad indicare la proprietà di essere condiviso da tutte le istanze della classe in cui è dichiarato.

Quindi le regole dell'incapsulamento quando impostiamo questo modificatore sono alterate.

Tecnicamente è possible incapsulare una variabile statica, ma quella variabile verrà comunque condivisa da tutti gli oggetti istanziati dalla stessa classe.

Quindi l'incapsulamento non si applica alle variabili statiche. Non è possibile applicare questo modificatore a variabili locali, parametri di metodi, costruttori o classi.

Anche senza istanziare la classe, l'utilizzo di un membro statico provocherà il caricamento in memoria della classe contenente il membro in questione che condividerà il ciclo di vita con quello della classe.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Un metodo statico essendo condiviso da tutti gli oggetti istanziati dalla classe in cui è dichiarato appartiene alla classe stessa e non ad un'istanza particolare.

Per questo motivo un membro statico ha la caratteristica di poter essere utilizzato mediante una sintassi del tipo:

NomeClasse.nomeMetodo (OK)

invece di

nomeOggetto.nomeMetodo (NO!)

Lo stesso vale per una variabile di istanza.

Un esempio di metodo statico è il metodo sqrt() della classe Math appartenente al package java.lang e che svolge la radice quadrata del numero fornito in input.

Come mostrato in Fig. 6 può essere chiamato tramite la sintassi Math.sqrt(numero).

Math è quindi il nome della classe e non il nome di un'istanza di quella classe (infatti si usa l'identificatore con la lettera maiuscola).

La ragione per cui la classe Math dichiara tutti i suoi metodi statici è facilmente comprensibile.

Trattandosi di operazioni matematiche, il funzionamento dipende dai parametri passati in ingresso piuttosto che dal valori di variabili di istanza. Infatti dichiarando due oggetti diversi obj1 e obj2 della classe Math, invocando i seguenti comandi:

obj1.sqrt(4);

obj2.sqrt(4);

si avrebbe lo stesso risultato ovvero 2. Questo perché sqrt calcola il risultato basandosi solo sull'input e non sugli attributi valorizzati di un particolare oggetto della classe Math.

Un metodo dichiarato static e public può considerarsi una sorta di funzione e non a caso la radice quadrata è definita come funzione matematica.

Per tale ragione questo tipo di approccio ai metodi è limitato solo ai casi in cui l'esecuzione del metodo non dipende dalle caratteristiche dell'oggetto su cui viene invocato il metodo.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
import java.lang.*;

public class MathDemo {

   public static void main(String[] args) {

      // get two double numbers numbers
      double x = 9;
      double y = 25;

      // print the square root of these doubles
      System.out.println(Math.sqrt(x));
      System.out.println(Math.sqrt(y));
   }
}
```

Fig. 6: Esempio di uso di un metodo statico sqrt

Un altro esempio di utilizzo di un metodo statico è mostrato in Fig. 7.

Per esempio un metodo dichiarato static correttamente potrebbe essere un metodo di una classe StringUtils chiamato contaOccorrenze e che conta quante occorrenze ci sono all'interno della stringa data string di un certo carattere specificato character.

In questi casi dichiarare il metodo static è certamente una buona pratica perché il risultato non dipende dallo stato dell'oggetto ma solo in questo caso dal parametro in ingresso.

Per invocare il metodo contaOccorrenze è stato adoperato il nome della classe (senza istanziarla), passato una stringa come primo parametro e come secondo un carattere da ricercare nella stringa e di cui si darà in output il numero di occorrenze nella stringa.

```
public class StringUtils {

   public static int contaOccorrenze(String string, char character) {
        int count = 0;
        for (var i=0; i<string.length(); i++) {
               if (string.charAt(i) == character) {
                      count++;
               }
        }
   return count;
   }
// alri metodi che fanno operazioni sulle stringhe
}
```

Fig. 7: Esempio di uso di un metodo statico contaOccorrenze

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Un esempio di variabile static è mostrato in Fig. 8.

Una variabile statica essendo condivisa da tutte le istanze della classe assumerà lo stesso valore per ogni oggetto di una classe.

Nell'esempio si dichiara e si usa una classe ClasseDiEsempio con una variabile a dichiarata static.

Prima di tutto si manda in stampa a tramite l'istruzione con la sintassi del tipo NomeClasse.nomeVariabile.

Il valore a video sarà 0. Infatti a differenza di una variabile d'istanza ordinaria, una variabile statica viene inizializzata al valore nullo del suo tipo quando viene caricata in memoria la classe e non al momento dell'istanza dell'oggetto: non è una variabile d'istanza. Dopodiché istanziamo due oggetti della classe ClasseDiEsempio, assegnamo il valore 10 alla variabile a dell'oggetto ogg1.

Stampiamo a video il valore di a sia dell'oggetto ogg1 che ogg2. Facciamo poi lo stesso valorizzando la variabile statica a dell'oggetto ogg2.

Come si può osservare se un'istanza modifica la variabile statica, essa risulterà modificata anche relativamente all'altra istanza.

Infatti essa è condivisa dalle due istanze e risiede proprio nella classe motivo per cui può essere riferita anche attraverso il nome della classe.

```
…
public class ClasseDiEsempio {
        public static int a=0; // condivisa da tutte le istanze
}

public class Principale {
        public static void main (String args[]) {
                System.out.println(''a= '' + ClasseDiEsempio.a); //stamperà 0

                ClasseDiEsempio ogg1 = new ClasseDiEsempio();
                ClasseDiEsempio ogg2 = new ClasseDiEsempio();
                ogg1.a=10;
                System.out.println(''ogg1.a= '' + ogg1.a); //stamperà 10
                System.out.println(''ogg2.a= '' + ogg2.a); //stamperà 10
                ogg2.a=20;
                System.out.println(''ogg1.a= '' + ogg1.a); //stamperà 20
                System.out.println(''ogg2.a= '' + ogg2.a); //stamperà 20

        }
}
```

Fig. 8: Esempio di variabile statica (aggiornamento del valore)

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Una variabile statica tipo potrebbe ad esempio essere utile per contare il numero di oggetti istanziati da una classe.

Un altro esempio è mostrato in Fig. 9 dove si può osservare come la variabile d'istanza number sia valorizzata nel costruttore di questa classe con il valore della variabile statica counter. In questo modo number rappresenterà proprio il numero seriale di ogni oggetto istanziato.

Se istanziamo prima c1 e poi c2, dopo aver istanziato c1 la variabile statica counter vale 1 e la variabile di istanza c1.number vale sempre 1. Dopo aver istanziato c2 allora la variabile statica counter vale 2.

Infatti essendo condivisa da tutte le istanze della classe non verrà azzerata nuovamente per ogni istanza.

Invece la variabile c1.number varrà sempre 1 mentre la variabile c2.number varrà 2.

```
…
public class Counter {
        private static int counter = 0 ; //inizializzazione superflua
        private int number;
        private Counter() {
                counter++;
                setNumber(counter);
        }
        public void setNumber (int number) {
                this.number = number;
        }
        public int getNumber (int number) {
                return number;
        }

}
…
Counter c1 = new Counter(); //counter vale 1 e c1.number vale 1
Counter c1 = new Counter(); //counter vale 2 e c2.number vale 2
```

Fig. 9: Esempio di variabile statica (per il conteggio istanze)

Una variabile statica già incontrata è System.out, una variabile statica della classe System (appartenente al package java.lang).

Il suo utilizzo è mostrato in Fig. 10. Si tratta di una variabile di tipo PrintStream (package java.io) che è un tipo di classe che permette di stampare (print) un flusso (stream) di dati su una certa destinazione (System.out) ed è questa classe che dichiara il metodo println utilizzato fino ad ora. In particolare l'oggetto System.out rappresenta l'output standard del sistema ovvero nel caso di Windows rappresenta il prompt dei comandi. Ecco perché una chiamata a System.out.println stampa il risultato sulla prompt dei comandi.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
…
public class Principale {
        public static void main (String args[]) {
                System.out.println(«Hello World»); //stamperà 0
        }
}
```

Fig. 10: Esempio di variabile statica (system.out)

Il modificatore static prescinde dal concetto di oggetto e lega strettamente le variabili al concetto di classe che a sua volta si innalza a qualcosa di più di un semplice mezzo per definire oggetti.

Per tale motivo a volte ci si riferisce alle variabili d'istanza statiche come variabili di classe.

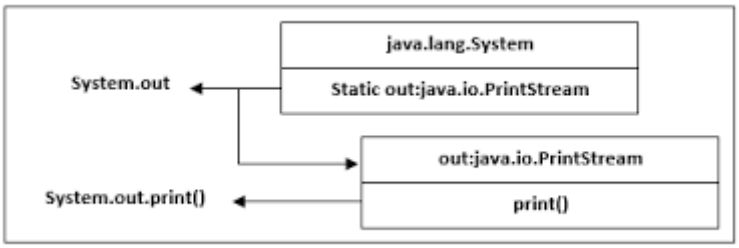


Fig. 11: Funzionamento di System.out [ref.: javatpoint.com]

E' importante fare un buon uso del modificatore static senza abusarne.

Usare static per dichiarare statici i metodi che non dipendono dallo stato dell'oggetto o sfruttare variabili statiche per mantenere il conto delle istanze.

Sicuramente occorre limitare l'uso di static per creare variabili globali quando abbinato a public.

Le variabili globali rappresentano una scorciatoia per i programmatori, ma sono difficili da gestire e per questo sono a ragione considerate esempio di cattiva progettazione.

Questo tipo di approccio alle variabili quindi va evitato. Tuttavia è però utile dichiarare costanti globali definite come public, static e final.

Per esempio la classe Math definisce due costanti statiche pubbliche; PI e E.

Se una variabile d'istanza è dichiarata final e inizializzata al momento della dichiarazione è opportuno dichiararla anche static per evitare che venga inizializzata ogni volta che si istanza un oggetto.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Bibliografia

- Il nuovo Java,, Claudio De Sio Cesari, Hoepli Informatica
- Programmazione di base e avanzata con Java, Walter Savitch, Pearson

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).